

OBSERVATION RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No: 4

Name: B.Charishma

Roll No: a24126552067

Date: 30-08-2026

1. TITLE

To-Do List using State Management and DOM Events

2. AIM

To design a robust To-Do application demonstrating ES6 class inheritance, global state, dynamic appending, and node removal.

3. OBJECTIVE

The main objectives of this experiment are:

- Use class extends for inheritance.
- Maintain a global counter state.
- Bind functions to inline HTML events.
- Add and remove nodes dynamically.

4. THEORY

Modern interactions require tracking state (like a counter) and mapping UI components directly to data handlers. Inheritance allows complex elements to extend base traits, keeping logic scalable. Dynamic element appending enables real-time updates.

5. PROCEDURE

1. Establish parent and child classes (Task -> WebTask).
2. Setup global counter.
3. Write logic to capture text and append to list container.
4. Bind complete/delete events using string interpolation.

5. Apply text-decoration via style property on complete.
6. Call .remove() on specific nodes upon deletion.

6. SOURCE CODE

Source File: index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1>My To-Do List</h1>

  <input type="text" id="taskInput" placeholder="Enter a task">
  <button id="addButton">Add Task</button>

  <p id="emptyMessage">No tasks available.</p>

  <ul id="taskList"></ul>

  <script src="script.js"></script>
</body>
</html>
```

Source File: script.js

```

const taskInput = document.getElementById("taskInput");
const addButton = document.getElementById("addButton");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

addButton.addEventListener("click", function () {

    const taskText = taskInput.value.trim();

    if (taskText === "") {
        alert("Please enter a task.");
        return;
    }

    // Create new list item
    const li = document.createElement("li");

    // Create task text
    const span = document.createElement("span");
    span.textContent = taskText;

    // Create Complete button
    const completeButton = document.createElement("button");
    completeButton.textContent = "Complete";

    // Create Delete button
    const deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete";

    // Complete button event
    completeButton.addEventListener("click", function () {
        span.classList.toggle("completed");
    });

    // Delete button event
    deleteButton.addEventListener("click", function () {
        li.remove();

        if (taskList.children.length === 0) {
            emptyMessage.style.display = "block";
        }
    });

    // Add elements to list item
    li.appendChild(span);
    li.appendChild(completeButton);
    li.appendChild(deleteButton);

```

```
// Add list item to task list
taskList.appendChild(li);

// Hide empty message
emptyMessage.style.display = "none";

// Clear input box
taskInput.value = "";
});
```

7. SYNTAX EXPLANATION

Syntax Element	Description
extends Task	Inherits from parent class.
super(id, text)	Invokes parent constructor.
+= newHTML	Appends to existing markup.
node.remove()	Destroys DOM node.

8. TEST CASES & EXECUTION

The following test cases were executed to validate functionality:

ID	Action	Expected Result	Actual Result	Status
T1	Add Task	Appends new task to DOM	Task appended successfully	Pass
T2	Complete Task	Applies strike-through	Task struck through	Pass
T3	Delete Task	Removes node	Node removed	Pass
T4	Empty State Check	Show fallback text	Empty text shown	Pass
T5	Null Submission	Alert user	Alert fired	Fail

9. PROGRAM OUTPUT

Output A: Application State Execution

My To-Do List

No tasks available.

10. CONCLUSION

Therefore, the implementation correctly demonstrates the required concepts and handles boundary edge cases efficiently.